

Plan-and-Avoid: C/C++ Implementation Manual

I. INTRODUCTION

This manual provides a concise description of the functions implemented in the accompanying C and C++ source code of Plan-and-Avoid framework, including their purpose and use within the software workflow. All explanatory text in this manual is written by generative AI softwares.

II. CONFLICT-AWARE TRAJECTORY PLANNING SOFTWARE

This section documents the Trajectory Planning software, which implements the *Plan* stage of the Plan-and-Avoid framework. The *Plan* stage generates a priority trajectory that minimizes avoidable conflicts while satisfying aircraft performance constraints and emergency mission objectives. It evaluates candidate trajectories with respect to ground risk, static airspace, and dynamic air traffic, producing both search-based and Dubins benchmark solutions together with detailed planning outputs for subsequent conflict-resolution, analysis, and visualization.

A. Directory Structure

The software is organized into modular components to separate application logic, reusable libraries, input data, intermediate build products, and generated outputs. Figure 1 illustrates the top-level directory structure.

The `src` directory contains the main application entry point (`plan.c`), which initializes the software and coordinates execution. Reusable functionality is implemented as independent modules under the `lib` directory. Each module follows a consistent structure consisting of `include` (public header files), `src` (source code), `obj` (compiled object files), and `data` (module-specific input files, when applicable).

Project-wide input datasets are stored under `data`, including aircraft parameters, census data, and MATLAB-generated datasets. Compiled object files are written to `obj`, executable outputs and results are stored in `out`, and temporary files are placed in `tmp`. The software is compiled using the top-level `Makefile`, while `ac1m.cfg` contains the primary runtime configuration parameters.

1) *data*: The `data` directory stores the external datasets and precomputed files required by the planning framework. Its contents are grouped into three categories: aircraft-specific data, census population data, and MATLAB data files for visualization of airspace and census datasets.

The `aircraft` directory contains Cessna 182 performance parameters and precomputed flight envelope information. The file `c182.dat` defines the Cessna 182 model parameters. The `gamma_straight` files and `gammaOpt.dat` store glide path angles obtained through brute-force searches under different wind conditions. The file `viableGammaTurn.bin`

contains forward-invariant glide path angles used to preserve airspeed.

The `census` directory contains Washington, D.C., census shapefiles and associated database and projection files. The datasets are retained to support population-based ground risk calculations. Each shapefile dataset may include `.shp`, `.shx`, `.dbf`, and `.prj` components describing geometry, indexing, attributes, and coordinate reference information, respectively. These data are used to create R*-Tree for population density query.

The `matlab` directory contains datasets generated or processed in MATLAB. These include the airspace traffic-density heatmap, dynamic ADS-B traffic data, candidate landing sites, airspace-corridor definitions, northbound static ADS-B traffic data, and census data in MATLAB format. These files are used to visualize environment and risk factors in MATLAB.

2) *out*: The `out` directory stores configuration snapshots, trajectory-planning results, search diagnostics, conflict predictions, and runtime logs generated during execution. Most outputs are written in CSV format to support post-processing and visualization.

The principal output files are summarized as follows:

- `actions.csv`: action sequence associated with the search-based solution.
- `path.csv`: state sequence defining the search-based solution path.
- `finalturn.csv`: final Dubins maneuver used to align the aircraft with the runway.
- `airspaceRisk0.csv`: discrete airspace-risk samples and corresponding risk metric values along the search-based path.
- `airspaceRisk1.csv`: discrete airspace-risk samples and corresponding risk metric values along the best Dubins path.
- `conflicts0.csv`: dynamic-traffic conflict prediction log for the search-based path, including position, time, and intruder identifier.
- `conflicts1.csv`: dynamic-traffic conflict prediction log for the best Dubins path, including position, time, and intruder identifier.
- `explored.csv`: states explored during the search process.
- `explored_actions.csv`: actions associated with the explored search states.

B. Build and Runtime Configuration

The software is compiled using the top-level `Makefile` and configured at runtime through `ac1m.cfg`. The `Makefile` defines the compilers, external-library paths, source files, object files, and final executable. The configuration file defines the planning scenario, aircraft model, risk parameters, traffic assumptions, and search settings.

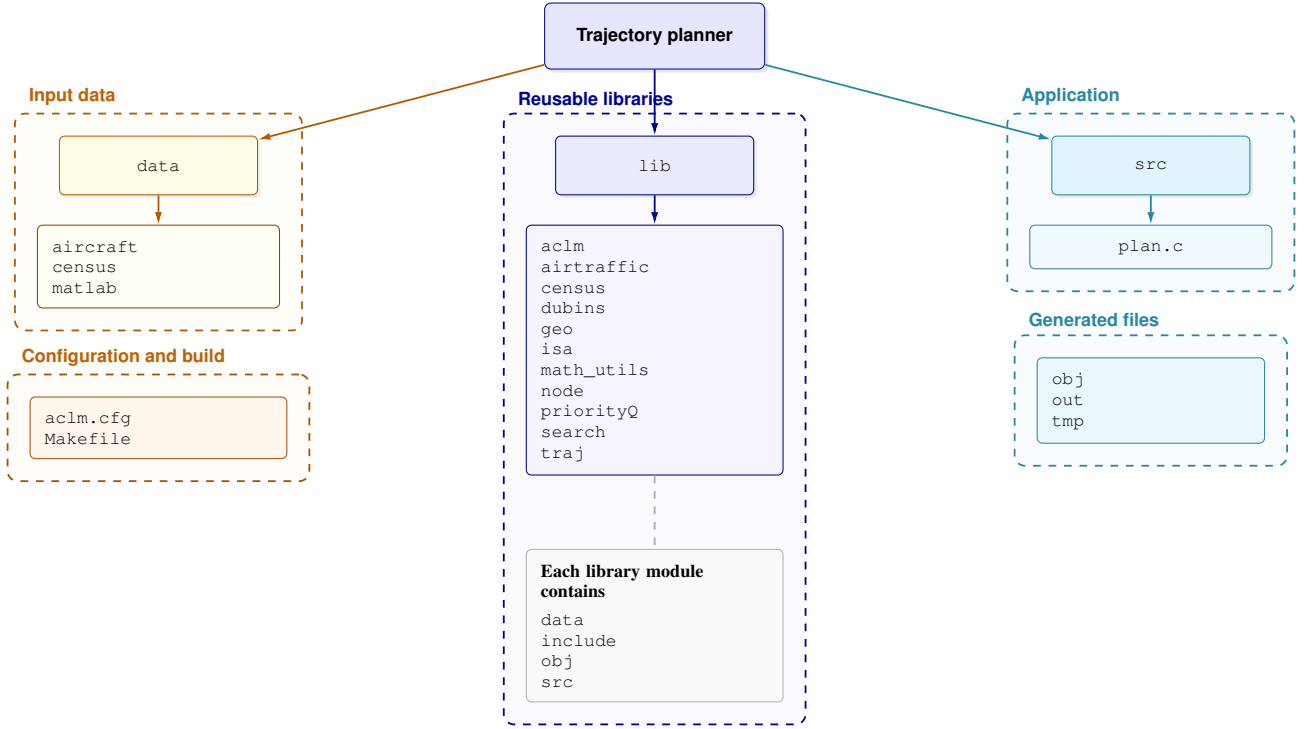


Fig. 1. Real-time ACLM software directory structure. Each library module follows a common organization containing source code, public headers, object files, and optional module-specific data.

1) *Makefile*: The default build command is

```
make
```

which creates the `plan` executable. Generated object files and the executable can be removed using

```
make clean
```

The current compiler and library paths correspond to a macOS Homebrew installation. Users must update the following entries to match their local system:

CC, CXX: Installed C and C++ compilers, such as `clang/clang++` or `gcc/g++`.

CFLAGS: Include directories for PROJ, libspatialindex, and NLOpt.

LDFLAGS: Library directories for Shapelib, PROJ, libspatialindex, and NLOpt.

-stdlib=libc++: Remove this option when it is unsupported by the selected compiler or operating system.

The internal source and object paths require no changes if the supplied directory structure is preserved. When a source file is added, removed, or relocated, the corresponding object-file entry and compilation rule must also be updated. Because the project includes a C++ wrapper, the final link command should preferably use `$(CXX)`.

2) *ac1m.cfg*: The file `ac1m.cfg` defines the runtime inputs for a planning case. It must remain in the project root unless the path supplied to the executable is changed. Lines beginning with `#` are comments.

a) *Solver Selection*: **SOLVER**: Selects the planning mode:

3: landing-site selection and search with dynamic traffic, static airspace, and ground risk.

b) *Emergency and Terminal States*: **INIT_LAT**,

INIT_LON: Initial emergency latitude and longitude [deg].

INIT_ALT: Initial MSL altitude [ft].

INIT_HDG: Initial aircraft heading [deg].

GOAL_LAT, GOAL_LON: Goal-state latitude and longitude [deg].

GOAL_ALT: Goal-state MSL altitude [ft].

GOAL_HDG: Goal-state heading [deg].

TD_LAT, TD_LON: Touchdown latitude and longitude [deg].

TD_ALT: Touchdown MSL altitude [ft].

TD_HDG: Runway or touchdown heading [deg].

The goal and touchdown entries are mandatory for solver types 0 and 1. For landing-site-selection modes, they may be overwritten by the selected site.

c) *Wind and Aircraft Parameters*: **WIND_SPEED**: Wind speed [kts].

WIND_DIR: Wind direction [deg].

TURN_RAD: Aircraft turn radius [ft].

GAMMA_BG_TURN: Best-glide turning flight-path angle [deg].

GAMMA_OPT_TURN: Nominal turning flight-path angle [deg].

AIRSPEED: Reference true airspeed [kts].

d) *Earth Model*: **EARTH_MODEL**: Selects the geodesic model:

0: WGS84.

1: Bessel ellipsoid.

2: International ellipsoid.

3: Flat-Earth model.

4: Spherical-Earth model.

Note: Planner is tested with WGS84 model, only.

e) *Risk Computation*: **ASRISK_W_HMAX**: Upper altitude bound for static-airspace weighting [ft].

ASRISK_W_HMIN: Lower altitude bound for static-airspace weighting [ft].

AIRSPACE_INTEG_DT: Static-airspace integration time step [s].

GROUND_INTEG_DT: Ground-risk integration time step [s].

GROUND_RISK_WEIGHT: Weight assigned to population risk. The complementary airspace-risk weight is computed internally.

CROSSOVER_H: MSL altitude below which population exposure is evaluated [ft].

MIN_POP, **MAX_POP**: Minimum and maximum population-density bounds used for normalization [persons/m²].

CENSUS_SHPFILE_DIR: Path to the census shapefile used for population-density queries.

The shapefile path must be updated when the census dataset is moved. Relative paths are interpreted from the project root.

f) *Dynamic Traffic*: **DYNAMIC_TRAFFIC_DATAFILE**: Name of the binary dynamic-traffic dataset stored under `lib/airtraffic/data`.

g) *Navigation and Separation Parameters*: **RNP_TER**: Terminal-area lateral navigation-performance bound [NM].

RNP_FIN: Final-approach lateral navigation-performance bound [NM].

IAF_ALT: Initial-approach-fix MSL altitude [ft].

ERR_ADSB_P: ADS-B horizontal position error [NM].

ERR_ADSB_V: ADS-B velocity error [m/s].

DELAY_ADSB: ADS-B or communication delay [s].

ERR_BARO_A5000: Barometric altitude-error bound at or above 5000 ft MSL [ft].

ERR_BARO_B5000: Barometric altitude-error bound below 5000 ft MSL [ft].

HORZ_CLEARANCE: Required horizontal separation [NM].

VERT_CLEARANCE: Required vertical separation [ft].

h) *Search Parameters*: **MAX_ITER**: Maximum number of expanded search states.

DISCRETIZE_HDG: Heading tolerance used for state clustering [deg].

DISCRETIZE_TIME: Time tolerance used for state clustering [s].

ALLOWED_HDG_CHG: Five allowable course changes [deg]. Negative values indicate left turns.

ADAPTIVE_SEGM_LEN_H: MSL altitude below which the minimum search-segment length is retained [ft].

III. CONFLICT-AWARE TRAJECTORY PLANNER SOURCE CODE REFERENCE

This section documents the source files that implement the Conflict-Aware GGS software. Figure 2 first summarizes the high-level execution flow and interactions among the principal modules, from scenario initialization and trajectory generation to shared ground- and airspace-risk evaluation and output generation. The remainder of the section describes each source file, summarizing its principal data structures and functions, its role in the overall planning workflow, and its primary points of use.

A. *plan.c*

The file `plan.c` contains the executable entry point and coordinates the complete contingency-planning workflow. It initializes the planning problem, executes the search-based planner, evaluates ground and airspace risk, generates a minimum-risk Dubins trajectory for comparison or fallback, records performance metrics, writes output files, and releases allocated resources.

1) *PathMetrics*: **Purpose**: Stores ground risk, airspace risk, flight-path-angle change, total runtime, and risk-evaluation runtime for either the search-based or Dubins trajectory.

Usage: Used by `main()` and `appendLogEntry()` to organize and record trajectory-performance metrics.

2) *ensureDirectoryExists()*: **Purpose**: Verifies that an output directory exists and creates it when necessary.

Usage: Called by `main()` before writing trajectory files and the summary log.

3) *unavailableMetrics()*: **Purpose**: Returns a `PathMetrics` structure initialized with NaN values to indicate unavailable results.

Usage: Called by `main()` to initialize search and Dubins metrics before trajectory generation and evaluation.

4) *appendLogEntry()*: **Purpose**: Appends the initial condition, planner exit flag, search metrics, Dubins metrics, and landing-site identifiers to `out/log.csv`.

Usage: Called by `main()` after each planning outcome, including successful, unreachable, and invalid-risk cases.

5) *getSelectedSiteIdent()*: **Purpose**: Returns the identifier of the landing site selected by the planner, or -1 when no valid selection is available.

Usage: Called by `main()` when writing the scenario result to the summary log.

6) *getHighestRankedSiteIdent()*: **Purpose**: Returns the identifier of the highest-ranked candidate landing site, or -1 when no ranked site is available.

Usage: Called by `main()` to compare the selected landing site with the highest-ranked candidate in the summary log.

7) *cleanupResources()*: **Purpose**: Releases the planning problem, Dubins workspace, and selected Dubins path while preventing double deallocation when the workspace and selected path refer to the same object.

Usage: Called by `main()` before each program exit.

8) *main()*: **Purpose**: Executes the complete contingency-planning application.

Usage: Serves as the program entry point and performs the following operations:

- 1) creates the output directory;
- 2) allocates and initializes the planning problem;
- 3) reads `aclm.cfg` and executes the search-based planner through `runEmergencyPlanning()`;
- 4) handles unreachable, invalid, or state-expansion-limit outcomes;
- 5) evaluates ground and airspace risk for the search-based trajectory;
- 6) computes a minimum-risk Dubins trajectory for benchmarking or fallback;

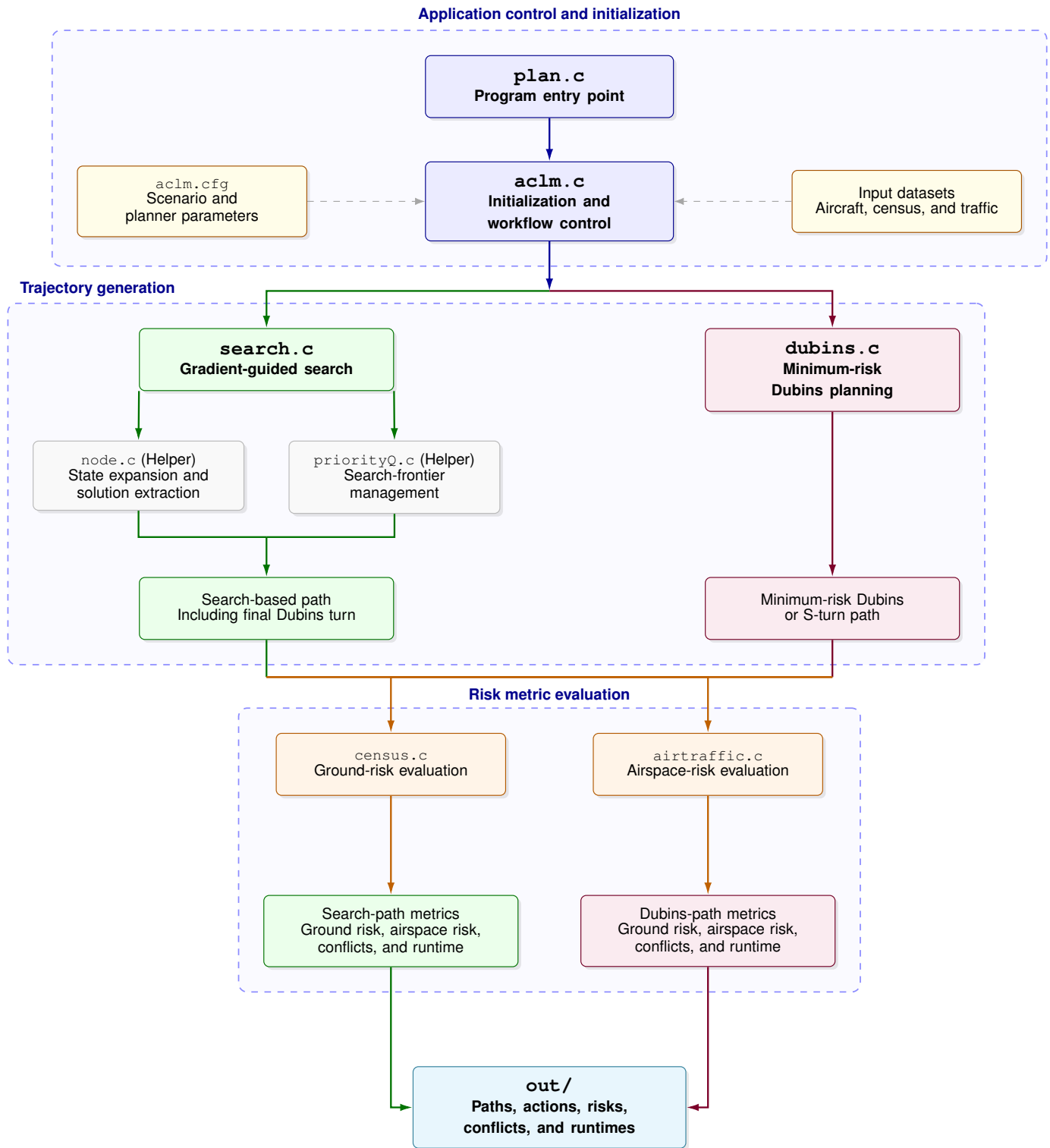


Fig. 2. High-level real-time ACLM execution flow. The search-based and minimum-risk Dubins trajectories are generated independently and evaluated using common ground- and airspace-risk modules before their metrics are recorded.

- 7) writes trajectory files and appends the scenario results to `out/log.csv`; and
- 8) releases all dynamically allocated resources.

The function interprets the planner exit flags as follows:

- -1: no reachable landing site or feasible fallback trajectory;
- -2: initialization in no-fly zone;
- -3: all candidate landing sites were exhausted;
- 0: the search and Dubins found solutions to the utility-wise best landing site
- 1: the search and Dubins found a solutions to an alternate landing site

- 2: the search reached its state-expansion limit and a Dubins fallback was attempted; and
- 4: the search found a solution, but the Dubins benchmark was unavailable or invalid.

B. *aclm.c*

The file *aclm.c* implements the high-level Assured Contingency Landing Management workflow and its supporting utilities. It initializes planning data, loads configuration and environmental datasets, ranks landing sites, computes glide parameters and trajectory risks, invokes the search and Dubins planners, manages output files, and releases dynamically allocated resources. :contentReference[oaicite:0]index=0

1) *runEmergencyPlanning()*: **Purpose:** Executes the complete emergency-planning workflow. It loads the configuration and datasets, ranks landing sites, and repeatedly runs the search planner over the ranked candidates until a solution is found or all candidates are exhausted.

Usage: Called by *main()* in *plan.c*. It serves as the principal interface between the executable and the ACLM library.

2) *setUpProblem()*: **Purpose:** Initializes the *SearchProblem* structure, loads aircraft, Earth-model, risk, and search parameters, initializes census and glide-angle datasets, and allocates the Dubins-related structures.

Usage: Called by *runEmergencyPlanning()* before loading traffic data and selecting a landing site.

3) *loadHeatmaps()*: **Purpose:** Allocates and loads the air-traffic-density, helicopter-route, and prohibited-airspace heatmaps.

Usage: Called by *runEmergencyPlanning()* during scenario initialization.

4) *loadInitialState()*: **Purpose:** Reads the initial aircraft latitude, longitude, altitude, and heading from the configuration file.

Usage: Called by *setUpProblem()*.

5) *loadGoalState()*: **Purpose:** Reads the goal-state position and heading from the configuration file.

Usage: Called by *setUpProblem()*. The values may later be updated for the selected landing site.

6) *loadTouchdownState()*: **Purpose:** Reads the touchdown position and runway heading from the configuration file.

Usage: Called by *setUpProblem()*.

7) *loadAircraftParams()*: **Purpose:** Loads the aircraft turn radius, glide-angle parameters, and airspeed, and converts airspeed from knots to feet per second.

Usage: Called by *setUpProblem()* after allocating the aircraft structure.

8) *loadEarthModelParams()*: **Purpose:** Loads the selected Earth model and the wind speed and direction.

Usage: Called by *setUpProblem()* before geodesic and glide calculations.

9) *loadRiskComputationParams()*: **Purpose:** Loads ground- and airspace-risk parameters, computes their corresponding weights and spatial integration steps, and returns the census-shapefile path.

Usage: Called by *setUpProblem()* before initializing the census dataset.

10) *loadSearchParams()*: **Purpose:** Loads the search iteration limit, heading and time discretization, allowable course changes, and adaptive segment-length threshold.

Usage: Called by *setUpProblem()*.

11) *loadDynamicTrafficParams()*: **Purpose:** Loads dynamic-traffic data paths, navigation-performance bounds, surveillance and barometric errors, communication delay, separation requirements, and approach altitude.

Usage: Called by *runEmergencyPlanning()* when the dynamic-traffic solver is enabled.

12) *getOptimalGamma()*: **Purpose:** Bilinearly interpolates the straight-flight best-glide, maximum-flap-extension, and reference-air-speed glide angles for the current wind speed and relative wind direction.

Usage: Called by *selectLandingSite()* and *initDistanceParams()* to determine wind-dependent straight-flight glide performance.

13) *getOptimalGammaTurn()*: **Purpose:** Interpolates the turning glide angle for the current wind condition, aircraft heading, and commanded heading change.

Usage: Used during node expansion or trajectory propagation to assign a wind-dependent turning flight-path angle.

14) *getViableGamma()*: **Purpose:** Interpolates the forward-invariant turning glide angle for the current wind condition, aircraft heading, and commanded heading change.

Usage: Used during search-state propagation to preserve feasible turning flight.

15) *readGammaStraight()*: **Purpose:** Reads the pre-computed straight-flight glide-angle tables for best-glide, maximum-flap-extension, and reference operating conditions.

Usage: Called by *setUpProblem()* to populate the aircraft glide-performance arrays.

16) *read_exact()*: **Purpose:** Reads an exact number of binary data elements and reports whether the operation was successful.

Usage: Internal helper used by *readGammaTurn()*.

17) *readGammaTurn()*: **Purpose:** Reads and validates the binary forward-invariant turning glide-angle dataset and stores it in the aircraft structure.

Usage: Called by *setUpProblem()*.

18) *readSolverType()*: **Purpose:** Reads the selected planning solver from the configuration file.

Usage: Called by *runEmergencyPlanning()* before loading solver-specific data.

19) *editCFG()*: **Purpose:** Copies the primary configuration file into the case-output directory while replacing the initial, goal, and touchdown states with the current scenario values.

Usage: Called by *runEmergencyPlanning()* for each ranked landing-site attempt.

20) *copyCFGfile()*: **Purpose:** Copies a configuration file between two locations without modifying its contents.

Usage: General file-management utility. It is not called within *aclm.c*.

21) *readLandingSites()*: **Purpose:** Reads the binary landing-site database and constructs a linked list containing runway position, dimensions, and usage attributes.

Usage: Called by *selectLandingSite()*.

22) *selectLandingSite()*: **Purpose:** Determines which landing sites are reachable, computes a utility score based on glide feasibility, runway geometry, airport type, and wind, and ranks the reachable candidates.

Usage: Called by *runEmergencyPlanning()* before iterating the search planner over candidate landing sites.

23) *cmp()*: **Purpose:** Compares two landing sites by utility in descending order.

Usage: Passed to *qsort()* by *selectLandingSite()*.

24) *initDistanceParams()*: **Purpose:** Initializes the touchdown and approach-fix states for a selected landing site and computes the applicable glide angles, altitude loss, and best-glide range.

Usage: Called by *runEmergencyPlanning()* before each search attempt.

25) *groundRisk()*: **Purpose:** Samples population density along either the search-based or Dubins path and integrates the normalized exposure using the trapezoidal rule.

Usage: Called by *main()* to evaluate ground risk when population-risk weighting is enabled.

26) *minRiskDubins()*: **Purpose:** Initializes a three-dimensional Dubins trajectory between the emergency state and goal and invokes the minimum-risk Dubins-path solver.

Usage: Called by *main()* for fallback planning or benchmarking against the search-based solution.

27) *_airspaceRisk()*: **Purpose:** Samples static airspace heatmaps along a trajectory and integrates traffic-density, helicopter-route, and prohibited-airspace costs.

Usage: Legacy static-airspace risk evaluator. The dynamic-traffic implementation is provided by *airspaceRisk()*.

28) *airspaceRisk()*: **Purpose:** Samples a trajectory at one-second intervals, evaluates the maximum closest-point-of-approach conflict cost against all dynamic intruders, and integrates the resulting airspace-risk metric.

Usage: Called by *main()* for both search-based and Dubins paths. It also writes risk samples and conflict-prediction logs.

29) *freePath()*: **Purpose:** Releases a path structure, including the node array and dynamically allocated actions.

Usage: General memory-management utility for path objects.

30) *freeHeatmap()*: **Purpose:** Releases an airspace heatmap and its grid and data arrays.

Usage: Called by *freeProblem()* for traffic, helicopter, and prohibited-airspace heatmaps.

31) *freeDynamicTraffic()*: **Purpose:** Releases the dynamic-traffic filename, flight array, individual trajectory-sample arrays, and enclosing structure.

Usage: Called by *freeProblem()*.

32) *freeRankedSites()*: **Purpose:** Releases the array of pointers used to rank landing sites.

Usage: General memory-management utility. It is not called within *aclm.c*.

33) *freeProblem()*: **Purpose:** Releases all memory owned by a *SearchProblem*, including aircraft data, census structures, search containers, paths, landing sites, heatmaps, dynamic traffic, and the problem structure itself.

Usage: Called through *cleanupResources()* in *plan.c* before program termination.

34) *printPointerAddress()*: **Purpose:** Prints the addresses of dynamically allocated members of a *SearchProblem* for debugging memory ownership and cleanup.

Usage: Debugging utility. Its calls in *freeProblem()* are currently disabled.

C. *airtraffic.c*

The file *airtraffic.c* implements dynamic-traffic loading, static airspace-heatmap queries, air-corridor geometry processing, trajectory timing, and loss-of-well-clear risk calculations. It supports both legacy corridor-based airspace evaluation and the dynamic multi-agent conflict metric used by the contingency planner. :contentReference[oaicite:0]index=0

1) *loadDynamicTraffic()*: **Purpose:** Reads the binary dynamic-traffic dataset, validates its format, allocates flight and sample arrays, and stores aircraft types, trajectories, groundspeeds, timestamps, and dataset statistics.

Usage: Called by *runEmergencyPlanning()* after dynamic-traffic parameters are loaded.

2) *readHeatMap()*: **Purpose:** Reads latitude, longitude, and altitude grids together with the associated three-dimensional airspace-cost array.

Usage: Called by *loadHeatmaps()* for traffic-density, helicopter-route, and prohibited-airspace datasets.

3) *getAirTrafficDensity()*: **Purpose:** Returns the heatmap value associated with a given latitude, longitude, and altitude.

Usage: Used by the static airspace-risk evaluator to sample traffic, helicopter, and prohibited-airspace costs.

4) *readAirCorridors()*: **Purpose:** Reads a text-based air-corridor database and constructs linked lists of corridor segments and their polyhedral surfaces.

Usage: Legacy air-corridor initialization utility used by corridor-based airspace evaluation.

5) *isInsidePolyhedron()*: **Purpose:** Determines whether an Earth-centered coordinate lies inside any air-corridor segment by testing it against the six inward-facing surface normals.

Usage: Called by *minimum_distance()* before computing the distance to corridor boundaries.

6) *doesOverlap()*: **Purpose:** Determines whether a point projects onto the interior of a quadrilateral corridor surface along the surface-normal direction.

Usage: Called by *minimum_distance()* to select between point-to-surface and point-to-edge distance calculations.

7) *point2vertexVec_to_edgeVector_angles()*: **Purpose:** Computes the angles between point-to-vertex vectors and the four edge vectors of a corridor surface.

Usage: Called by *minimum_distance()* to determine whether the nearest boundary feature is an edge or vertex.

8) *minimum_distance()*: **Purpose:** Computes the minimum three-dimensional distance from an aircraft position to all corridor polyhedra.

Usage: Called by *airCorridorOccupationCost()*.

9) *airCorridorOccupationCost()*: **Purpose:** Converts the minimum distance to an air corridor into a normalized occupation cost using a 500-ft buffer.

Usage: Legacy corridor-based airspace-risk utility.

10) *groundspeed()*: **Purpose:** Computes aircraft groundspeed along the current heading using true airspeed and the along-track wind component.

Usage: Called by *traversalDeltat()* and other trajectory-timing routines.

11) *traversalDeltat()*: **Purpose:** Computes the integer traversal time of a search action from its length and local groundspeed.

Usage: Used during search-node propagation and time indexing.

12) *VCPA()*: **Purpose:** Computes uncertainty-adjusted vertical separation from a selected intruder and identifies whether the encounter is vertically active.

Usage: Called by *JCPA()* before horizontal separation is evaluated.

13) *CPACost()*: **Purpose:** Converts horizontal and vertical separation deficiencies into a continuous loss-of-well-clear penalty.

Usage: Called by *JCPA()* after uncertainty-adjusted separation distances are obtained.

14) *JCPA()*: **Purpose:** Computes the conflict penalty between an ownship state and one dynamic intruder at a specified time index. It first checks vertical activity, then evaluates uncertainty-adjusted horizontal separation and returns the corresponding closest-point-of-approach cost.

Usage: Called by *airspaceRisk()* and during search-action risk evaluation.

D. *census.c*

The file *census.c* implements census-shapefile loading, spatial indexing, point-in-polygon queries, population-density lookup, and census-resource cleanup. It supports both direct polygon searches and accelerated R-tree queries used in ground-risk evaluation.

1) *initCensus()*: **Purpose:** Opens the census shapefile and attribute database, identifies the population and land-area fields, loads polygon geometries, and inserts their bounding boxes into an R-tree.

Usage: Called by *setUpProblem()* to initialize the census dataset used by ground-risk calculations.

2) *isPointInPolygon()*: **Purpose:** Determines whether a query point lies inside a polygon using the ray-casting algorithm.

Usage: Called by *getPopulationDensityLin()*.

3) *getPopulationDensityLinearSearch()*:

Purpose: Sequentially searches all census polygons, identifies the polygon containing the query location, and returns its capped population density.

Usage: Provides a reference or fallback population-density query without spatial indexing.

4) *freeCensus()*: **Purpose:** Closes shapefile and database handles, destroys loaded polygon objects, releases the polygon array, destroys the R-tree, and frees the census structure.

Usage: Called by *freeProblem()*.

5) *pointInPolygon()*: **Purpose:** Tests whether a point lies inside a multipart shapefile polygon using ray casting across all polygon parts.

Usage: Called by *findPolygon()* to verify R-tree candidates.

6) *findPolygon()*: **Purpose:** Queries the R-tree for polygons whose bounding boxes contain the specified point and returns the identifier of the polygon that geometrically contains it.

Usage: Called by *getPopulationDensityRTree()*.

7) *getPopulationDensityRTree()*: **Purpose:** Uses the R-tree and point-in-polygon test to identify the containing census polygon and returns its capped population density.

Usage: Called by *groundRisk()* for each trajectory sample.

8) *censusBoundingBox()*: **Purpose:** Returns the latitude and longitude bounds of the loaded census dataset.

Usage: General utility for validating or visualizing the census coverage region.

E. *rtree_wrapper.cpp*

The file *rtree_wrapper.cpp* provides a C-compatible interface to the C++ *libspatialindex* R*-tree implementation. It creates an in-memory spatial index, inserts two-dimensional bounding boxes, performs intersection queries, and returns matching identifiers to the C census module.

1) *RTreeIndex*: **Purpose:** Encapsulates the in-memory storage manager and R*-tree index provided by *libspatialindex*.

Usage: Instantiated once through the global pointer *g_rtree* and accessed by the C wrapper functions.

2) *RTreeIndex::RTreeIndex()*: **Purpose:** Creates an in-memory storage manager and initializes a two-dimensional R*-tree with the configured fill factor and node capacities.

Usage: Called by *rtree_init()* when the global index is first created.

3) *RTreeIndex::~~RTreeIndex()*: **Purpose:** Releases the R*-tree and its storage manager.

Usage: Invoked when *rtree_destroy()* deletes the global index.

4) *RTreeIndex::insert()*: **Purpose:** Inserts a two-dimensional bounding box and its associated integer identifier into the R*-tree.

Usage: Called by *rtree_insert()* while census polygons are indexed.

5) *RTreeIndex::search()*: **Purpose:** Performs an intersection query over a rectangular region and stores the identifiers of all matching entries.

Usage: Called by *rtree_search()* during census point queries.

6) *MyVisitor*: **Purpose:** Implements the *libspatialindex* visitor interface and collects the identifiers of entries returned by a spatial query.

Usage: Created internally by *RTreeIndex::search()*.

7) *MyVisitor::visitNode()*: **Purpose:** Satisfies the visitor interface for tree-node visits without performing additional processing.

Usage: Called internally by *libspatialindex* during a query.

8) *MyVisitor::visitData()*: **Purpose:** Appends the identifier of each matching spatial entry to the result vector.

Usage: Called internally by *libspatialindex* for each query result.

9) *MyVisitor::visitData(std::vector<...>)*: **Purpose:** Provides the batch-data visitor overload required by the interface; no processing is performed.

Usage: Interface-compliance method that is not otherwise used directly.

10) *rtree_init()*: **Purpose:** Creates the global R*-tree instance if it has not already been initialized.

Usage: Called by *initCensus()* before census-polygon bounding boxes are inserted.

11) *rtree_insert()*: **Purpose:** Inserts a bounding box and identifier into the global R*-tree.

Usage: Called by *initCensus()* for each census polygon.

12) *rtree_search()*: **Purpose:** Queries the global R*-tree, copies up to *max_results* matching identifiers into a C array, and returns the number of copied results.

Usage: Called by *findPolygon()* to identify candidate census polygons containing a query point.

13) *rtree_destroy()*: **Purpose:** Deletes the global R*-tree instance and clears its pointer.

Usage: Called by *freeCensus()* during census-resource cleanup.

F. *dubins.c*

The file *dubins.c* implements the three-dimensional Dubins-path solver and its supporting trajectory-generation utilities. It constructs Dubins path families, selects shortest or minimum-risk candidates, generates S-turn extensions when additional altitude dissipation is required, adjusts glide angles for wind-dependent feasibility, samples trajectories in space or time, and evaluates path risk.

1) *Dubins()*: **Purpose:** Constructs a specified Dubins path by computing turn-circle centers, tangent points, intermediate waypoints, path headings, turn directions, and total horizontal distance.

Usage: Called by *shortestDubins()*, *getMinRiskDubinsPath()*, and *getDubinsWithType()*.

2) *shortestDubins()*: **Purpose:** Evaluates the four turn-straight-turn Dubins families and retains the feasible path with minimum horizontal length.

Usage: Called during S-turn construction and other operations that require the shortest geometric Dubins connection.

3) *getIntermediateWaypoint()*: **Purpose:** Computes a laterally displaced intermediate waypoint used to form an S-turn path.

Usage: Called by *computeSturnDubins()*.

4) *computeSturnDubins()*: **Purpose:** Constructs two connected Dubins paths through an intermediate waypoint and iteratively adjusts the S-turn geometry until the target altitude loss is achieved.

Usage: Called by *getBestSturnPath()* for left- and right-side extensions.

5) *updateOptimalGammaTurn()*: **Purpose:** Updates the glide angles of the initial and final turning segments using wind-dependent turning-performance data. Large heading changes are divided into smaller increments and averaged.

Usage: Called during S-turn generation and minimum-risk Dubins evaluation before recomputing path geometry.

6) *getDubinsCoordinates()*: **Purpose:** Samples a Dubins trajectory at approximately uniform spatial intervals and optionally writes the resulting position and heading sequence to a CSV file.

Usage: Called by ground- and static-airspace-risk calculations.

7) *getDubinsCoordinatesWithFixedTimeStep_()*: **Purpose:** Samples a Dubins trajectory at one-second intervals using segment-average groundspeeds.

Usage: Earlier fixed-time-step sampling implementation retained as an internal or legacy utility.

8) *getDubinsCoordinatesWithFixedTimeStep()*: **Purpose:** Samples a Dubins trajectory at one-second intervals while preserving temporal continuity across orbit-straight transitions and connected S-turn segments.

Usage: Called by *airspaceRisk()* to align ownship trajectory samples with dynamic-traffic time indices.

9) *getBestSturnPath()*: **Purpose:** Generates left- and right-extended S-turn candidates, checks altitude feasibility, evaluates their risks, and returns the minimum-risk feasible option.

Usage: Called by *getMinRiskDubinsPath()* when a direct Dubins path does not dissipate sufficient altitude.

10) *evaluateDubinsRisk()*: **Purpose:** Computes ground risk, airspace risk, and weighted total risk for a Dubins or S-turn trajectory.

Usage: Called by *getBestSturnPath()* and *getMinRiskDubinsPath()*.

11) *getMinRiskDubinsPath()*: **Purpose:** Evaluates the four principal Dubins path families, adjusts turning glide angles, introduces S-turns when additional altitude loss is needed, reduces straight-segment slope when altitude loss is excessive, and returns the feasible path with minimum total risk.

Usage: Called by *minRiskDubins()* for fallback planning and benchmarking.

12) *reduceSlope()*: **Purpose:** Reduces the straight-segment glide angle so that the Dubins path reaches the required final altitude while remaining within the feasible straight-flight glide-angle range.

Usage: Called by *getMinRiskDubinsPath()* when the original path loses too much altitude.

13) *moveWaypoint()*: **Purpose:** Moves the terminal waypoint backward along the runway heading and adjusts its altitude to satisfy the minimum geometric separation required for Dubins construction.

Usage: Called by `getMinRiskDubinsPath()` when the initial and final waypoints are too close.

14) `getDubinsWithType()`: **Purpose:** Computes a Dubins path for a user-specified path family and copies the resulting geometry into the supplied path structure.

Usage: General utility for explicitly selecting a Dubins type.

15) `dubins_optimal()`: **Purpose:** Computes the internally optimized Dubins solution included from `dubins_optimal.inc`.

Usage: Called by `Dubins()` when the requested path type is `OPT`.

G. *geo.c*

The file `geo.c` implements the geodesic and coordinate-transformation utilities used throughout the planner. It supports flat-Earth, spherical-Earth, and Vincenty-based ellipsoidal calculations for distance, heading, position propagation, turn geometry, and conversion between geodetic and Cartesian reference frames.

1) `geo_dist()`: **Purpose:** Computes the horizontal distance and initial heading between two geodetic positions using the selected Earth model.

Usage: Used throughout the planner for path geometry, landing-site evaluation, trajectory construction, and separation calculations.

2) `geo_npos()`: **Purpose:** Computes a new geodetic position from an initial position, travel distance, and heading.

Usage: Used for waypoint propagation, Dubins-circle construction, approach-fix generation, and trajectory sampling.

3) `geo_normhdg()`: **Purpose:** Normalizes a heading to the interval $[0, 360)$ deg.

Usage: Called by `geo_dist()` and other heading-dependent geometry routines.

4) `geo_npost()`: **Purpose:** Computes the terminal position of a constant-radius turn from the initial heading, turn radius, and heading change.

Usage: Used by turn-trajectory calculations and waypoint propagation.

5) `geo_distt()`: **Purpose:** Computes the turn radius required to connect two positions given the initial heading.

Usage: Used by trajectory-geometry routines that infer circular-turn parameters.

6) `geo_npos3D()`: **Purpose:** Propagates a position horizontally and updates altitude using a specified flight-path angle.

Usage: Used for three-dimensional trajectory and search-space propagation.

7) `geo_earth curvature()`: **Purpose:** Computes the local Earth curvature radius at a specified geodetic latitude.

Usage: Called by `geo_lla2ecef()` and other Earth-centered coordinate calculations.

8) `geo_lla2ecef()`: **Purpose:** Converts latitude, longitude, and altitude into Earth-Centered, Earth-Fixed coordinates.

Usage: Used by local-frame transformations and legacy corridor-distance calculations.

9) `geo_lla2ned()`: **Purpose:** Converts a geodetic query position into a local North-East-Down frame referenced to another geodetic position.

Usage: Used for local relative-position calculations and Cartesian geometric analysis.

H. *math_utils.c*

The file `math_utils.c` implements general mathematical, geometric, and path-processing utilities used by the planner. Its functions support vector operations, corridor-distance calculations, angle normalization, search-space estimation, Dubins-based distance heuristics, path resampling, and auxiliary numerical operations.

1) `arr_length()`: **Purpose:** Returns the number of elements in a sentinel-terminated array of doubles.

Usage: Called by `minimum()`.

2) `flipplr()`: **Purpose:** Reverses the order of the four vertices defining a surface.

Usage: Called by `minimum_distance()` to correct surface orientation before geometric distance calculations.

3) `minimum()`: **Purpose:** Returns the minimum value in a sentinel-terminated array.

Usage: General numerical utility.

4) `dot()`: **Purpose:** Computes the dot product of two three-dimensional vectors.

Usage: Used by vector normalization, angle, surface, and polyhedron calculations.

5) `norm()`: **Purpose:** Computes the Euclidean norm of a three-dimensional vector.

Usage: Used by normalization and point-to-edge or point-to-vertex distance calculations.

6) `cross()`: **Purpose:** Computes the cross product of two three-dimensional vectors.

Usage: Used to obtain surface normals and point-to-edge distances.

7) `normalize()`: **Purpose:** Scales a three-dimensional vector to unit length when its magnitude is nonzero.

Usage: General geometric utility.

8) `createVector()`: **Purpose:** Constructs the displacement vector between two three-dimensional points.

Usage: Used throughout the corridor-overlap and distance calculations.

9) `angle_between_vectors()`: **Purpose:** Computes the angle between two three-dimensional vectors.

Usage: Called by `point2vertexVec_to_edgeVector_angles`

10) `surfaceCentroid()`: **Purpose:** Computes the centroid of a quadrilateral surface.

Usage: Called by `isInsidePolyhedron()`.

11) `surfaceNormal()`: **Purpose:** Computes the unit normal vector of a quadrilateral surface.

Usage: Called by `isInsidePolyhedron()`, `doesOverlap()`, and related surface-distance routines.

12) `surfaceEquation()`: **Purpose:** Computes the constant term of the plane equation associated with a surface.

Usage: Called by `distance_point2surface()`.

13) *distance_point2surface()*: **Purpose:** Computes the perpendicular distance from a point to a surface plane.

Usage: Called by *minimum_distance()* when the point projection lies within the surface.

14) *distance_point2edge()*: **Purpose:** Computes the shortest distance from a point to the line defined by a surface edge.

Usage: Called by *minimum_distance()* when the nearest corridor feature is an edge.

15) *distance_point2vertex()*: **Purpose:** Computes the minimum Euclidean distance from a point to the four vertices of a surface.

Usage: Called by *minimum_distance()* when no surface or edge projection is valid.

16) *wrapTo360()*: **Purpose:** Wraps an angle to the interval $[0, 360)$ deg.

Usage: Used throughout the planner for heading and wind-direction calculations.

17) *wrapTo180()*: **Purpose:** Wraps an angle to the interval $[-180, 180)$ deg.

Usage: Used for relative heading, wind-angle, and turn-direction calculations.

18) *size()*: **Purpose:** Estimates an upper bound on the number of discrete search states from altitude, spatial, and heading discretization parameters.

Usage: Used for search-complexity analysis or memory sizing.

19) *minDistance()*: **Purpose:** Estimates the minimum feasible path distance between two states using great-circle distance for nearby states and the shortest Dubins path otherwise.

Usage: Used as a distance heuristic or lower-level reachability estimate in the search planner.

20) *min()*: **Purpose:** Returns the smaller of two scalar values.

Usage: General numerical utility.

21) *max()*: **Purpose:** Returns the larger of two scalar values.

Usage: General numerical utility, including trajectory sample-count calculations.

22) *normalVector()*: **Purpose:** Computes a two-dimensional unit vector aligned with the heading of a specified state.

Usage: Used to define the goal-state normal for final-approach and solution-identification checks.

23) *cumulativeDistances()*: **Purpose:** Computes cumulative three-dimensional distance along the search path and from its final node to the goal state.

Usage: Called before equidistant trajectory resampling.

24) *linearInterpld()*: **Purpose:** Performs scalar linear interpolation between two data points.

Usage: Called by *sampleEquidistantPoints()*.

25) *sampleEquidistantPoints()*: **Purpose:** Resamples the search path at approximately uniform three-dimensional distance intervals using linear interpolation.

Usage: Used to generate spatially uniform path samples for risk integration or output processing.

26) *random_double()*: **Purpose:** Returns a uniformly distributed random value within a specified interval.

Usage: General utility for randomized scenario generation or testing.

I. node.c

The file *node.c* implements the creation, expansion, storage, extraction, and cleanup of search nodes and solution paths. It manages node actions, parent-child relationships, explored-state storage, child-state generation, cumulative timing and cost evaluation, and conversion of a terminal search node into path and action arrays.

1) *createAction()*: **Purpose:** Allocates and initializes a search action with heading change, segment length, and flight-path angle.

Usage: Used when constructing the discrete action set for node expansion.

2) *createNode()*: **Purpose:** Allocates a search node, copies its state and action, assigns its parent, and initializes cumulative path length and depth.

Usage: Called by *childNode()* and during search initialization.

3) *createClosedList()*: **Purpose:** Allocates the explored-state list with an initial storage capacity.

Usage: Called during search initialization.

4) *clInsert()*: **Purpose:** Appends a node to the explored-state list and doubles the storage capacity when necessary.

Usage: Called by the search algorithm after a node is expanded or closed.

5) *destroyNode()*: **Purpose:** Releases a node and its dynamically allocated action.

Usage: Called by *destroyTree()* and during failed child-expansion cleanup.

6) *destroyTree()*: **Purpose:** Traverses the parent chain from a node to the root and releases each node.

Usage: Used to clean up a temporary search branch.

7) *expandNode()*: **Purpose:** Generates the applicable action set and creates one child node for each action.

Usage: Called by the search algorithm when expanding a node.

8) *childNode()*: **Purpose:** Propagates a parent state through a selected action, creates the resulting child node, updates its timestamp, and evaluates its state cost.

Usage: Called by *expandNode()*.

9) *solution()*: **Purpose:** Extracts the root-to-goal node sequence, applies any altitude corrections, and returns a deep-copied *Path* structure.

Usage: Called after the search reaches a valid solution state.

10) *solutionActions()*: **Purpose:** Extracts the action sequence associated with a solution path and applies the final straight-flight glide-angle correction.

Usage: Called after solution identification to produce the executable action list.

11) *path()*: **Purpose:** Traverses parent pointers from a terminal node to the root and returns the node sequence in forward order.

Usage: Called by *solution()* and *solutionActions()*.

12) *destroyClosedList()*: **Purpose:** Releases all actions stored in the explored-state list, followed by the node array and list structure.

Usage: Called when restarting the search for another landing site and by *freeProblem()* during final cleanup.

J. *priorityQ.c*

The file *priorityQ.c* implements the binary-heap priority queue used by the search algorithm. It supports both minimum- and maximum-priority behavior, dynamic capacity growth, insertion, removal, priority updates, and extraction of the next node to expand.

1) *swap()*: **Purpose:** Exchanges two entries in the heap.

Usage: Called by *heapifyUp()* and *heapifyDown()*.

2) *compare()*: **Purpose:** Compares two priority values according to whether the queue is configured as a minimum or maximum heap.

Usage: Called by *heapifyUp()* and *heapifyDown()*.

3) *heapifyUp()*: **Purpose:** Restores the heap ordering by moving a newly inserted or updated entry toward the root.

Usage: Called by *pqInsert()* and *pqUpdatePriority()*.

4) *heapifyDown()*: **Purpose:** Restores the heap ordering by moving an entry downward after removal or replacement.

Usage: Called by *pqPop()*, *pqRemove()*, and *pqUpdatePriority()*.

5) *createPriorityQueue()*: **Purpose:** Allocates and initializes a binary-heap priority queue with the requested capacity and heap type.

Usage: Called during search initialization.

6) *destroyPriorityQueue()*: **Purpose:** Releases the queued nodes, heap storage, and priority-queue structure.

Usage: Called when restarting the search for another landing site and by *freeProblem()*.

7) *pqInsert()*: **Purpose:** Inserts a search node into the queue, expands the heap storage when necessary, and restores heap order.

Usage: Called by the search algorithm when adding candidate nodes to the open set.

8) *pqPop()*: **Purpose:** Removes and returns the highest-priority node from the queue.

Usage: Called by the search algorithm to select the next node for expansion.

9) *pqIsEmpty()*: **Purpose:** Reports whether the priority queue contains no nodes.

Usage: Used by the search loop as a termination check.

10) *pqContains()*: **Purpose:** Determines whether a node matching a supplied key is already present in the queue.

Usage: Used for open-set duplicate detection.

11) *pqRemove()*: **Purpose:** Removes the first queue entry matching a supplied key and restores heap ordering.

Usage: Used when replacing or discarding an existing open-set node.

12) *pqUpdatePriority()*: **Purpose:** Updates the priority of a matching queue entry and reestablishes heap order.

Usage: Used when a better path to an existing open-set node is found.

K. *search.c*

The file *search.c* implements the gradient-guided search algorithm and its principal cost, feasibility, path-correction, and output routines. It initializes and executes the search, generates feasible actions, propagates aircraft states, evaluates heuristic and cumulative risk costs, identifies valid terminal states, suppresses duplicate states, samples completed trajectories, and writes planning results.

1) *runSearch()*: **Purpose:** Executes the main search loop by repeatedly extracting the lowest-cost node, checking the goal condition, expanding feasible children, and inserting nonduplicate states into the frontier.

Usage: Called by *runEmergencyPlanning()* for each ranked landing-site candidate.

2) *initializeSearch()*: **Purpose:** Initializes search-distance parameters, cost normalizers, the priority queue, the explored-state list, and the initial search node.

Usage: Called by *runEmergencyPlanning()* before each search attempt.

3) *getActions()*: **Purpose:** Generates the five candidate search actions, including heading change, adaptive segment length, straight-flight glide angle, and turn glide angle.

Usage: Called by *expandNode()*.

4) *result()*: **Purpose:** Propagates a state through a selected action and computes the resulting position, heading, and altitude while accounting for straight and half-turn altitude losses.

Usage: Called by *childNode()*.

5) *courseCostNormalization()*: **Purpose:** Evaluates the course-alignment cost over a grid surrounding the reachable region and stores the minimum and maximum values used for normalization.

Usage: Called by *initializeSearch()*.

6) *populationCostNormalizer()*: **Purpose:** Computes the normalization factor used by the segment-level population-risk cost.

Usage: Called by *initializeSearch()*.

7) *distanceCost()*: **Purpose:** Computes the two distance-related heuristic terms from the traveled path length, minimum remaining distance, and optimum glide distance.

Usage: Called by *stateCost()*.

8) *courseCost()*: **Purpose:** Computes the normalized terminal-course alignment cost of a state relative to the goal-state normal vector.

Usage: Called by *stateCost()* and checked by *goalTest()*.

9) *populationCost()*: **Purpose:** Samples census density along a search action and integrates altitude- and distance-weighted population exposure.

Usage: Called by *stateCost()* when ground-risk weighting is enabled.

10) *altitudeWeight()*: **Purpose:** Computes the altitude-dependent weighting applied to static airspace cost near the goal.

Usage: Called by *airspaceCost()* and *maxDensityAhead()*.

11) *airspaceCost()*: **Purpose:** Samples static traffic, helicopter-route, and prohibited-airspace heatmaps along an action and integrates the resulting airspace cost.

Usage: Static-airspace segment-cost utility retained for heatmap-based planning.

12) *maxDensityAhead()*: **Purpose:** Samples a forward cone across multiple headings and glide levels to estimate nearby dynamic-traffic conflict density.

Usage: Called by *getActions()* for adaptive segment sizing and by *stateCost()* as a look-ahead heuristic.

13) *dynamicAirspaceCost()*: **Purpose:** Samples a search segment at one-second intervals and integrates the maximum conflict cost over all dynamic intruders.

Usage: Called by *stateCost()* to update cumulative airspace risk.

14) *finalTurnDynamicTrafficCost()*: **Purpose:** Samples the final Dubins turn against dynamic traffic and verifies that its average conflict cost remains below the specified threshold.

Usage: Optional final-turn feasibility check in *goalTest()*; the call is currently disabled.

15) *stateCost()*: **Purpose:** Computes the total node priority from cumulative dynamic-traffic risk and weighted heuristic terms, while rejecting unreachable or prohibited states.

Usage: Called for the initial node and each generated child node.

16) *dubinsFinalTurn()*: **Purpose:** Constructs and validates the final Dubins maneuver connecting a candidate search state to the runway approach and touchdown geometry.

Usage: Called by *goalTest()*.

17) *goalTest()*: **Purpose:** Determines whether a search node can form a valid terminal solution by checking course alignment, distance, altitude, final-turn feasibility, and remaining approach glide angle.

Usage: Called by *runSearch()* and during search initialization.

18) *modifyGamma()*: **Purpose:** Determines whether a uniform flight-path-angle adjustment can remove an altitude mismatch while keeping all path segments within feasible glide limits.

Usage: Auxiliary path-correction routine for terminal altitude matching.

19) *modifyGammaWind()*: **Purpose:** Iteratively computes wind-dependent segment glide-angle adjustments and cumulative altitude corrections required to eliminate a terminal altitude deficit.

Usage: Auxiliary path-correction routine for wind-aware altitude matching.

20) *isInHexagon()*: **Purpose:** Determines whether one state lies inside the hexagonal spatial cell centered on another state.

Usage: Called by *doesCluster()* and *doesClusterFrontier()*.

21) *doesCluster()*: **Purpose:** Checks whether a candidate node is equivalent to an explored state in time, heading, altitude, and horizontal position.

Usage: Called by *runSearch()* before inserting a child into the frontier.

22) *doesClusterFrontier()*: **Purpose:** Checks whether a candidate node is equivalent to a state already stored in the frontier.

Usage: Called by *runSearch()* to suppress duplicate open-set states.

23) *searchIntegrationCoordinates()*:

Purpose: Resamples the completed search path at approximately uniform spatial intervals.

Usage: Called by ground- and static-airspace-risk evaluation.

24) *searchIntegrationCoordinatesWithFixedTimeStep*

Purpose: Resamples the completed search path at fixed time intervals using segment-average groundspeed.

Usage: Called by *airspaceRisk()* to align search-path samples with dynamic-traffic time steps.

25) *computeCorrectedAltitudes()*: **Purpose:** Recomputes physically consistent waypoint altitudes by assigning entry and exit half-turn losses to the appropriate path segments.

Usage: Called by *dubinsFinalTurn()* and *writeResults()*.

26) *writeResults()*: **Purpose:** Writes search paths, action sequences, final-turn trajectories, minimum-risk Dubins paths, simulation geometry, and explored-state data to CSV files.

Usage: Called by *main()* after a valid search or fallback Dubins solution is available.

L. traj.c

The file *traj.c* implements low-level trajectory data handling and geometric evaluation. It initializes and copies linked trajectory segments, computes horizontal and angular distances, propagates altitude along three-dimensional paths, generates dense trajectory samples, evaluates time spent within altitude layers, and writes trajectory data for inspection or simulation.

1) *Traj_Copy()*: **Purpose:** Copies one trajectory segment, including waypoint, turn, distance, and aerodynamic attributes.

Usage: Called by *Traj_CopyAll()* and Dubins-path utilities.

2) *Traj_CopyAll()*: **Purpose:** Copies all segments in a linked trajectory sequence.

Usage: Used extensively when constructing temporary and candidate Dubins paths.

3) *Traj_InitArray()*: **Purpose:** Initializes a fixed-size array of trajectory segments, links adjacent entries, and assigns undefined values to unset fields.

Usage: Called whenever a new Dubins or trajectory structure is created.

4) *Traj_HDist()*: **Purpose:** Returns the total horizontal distance of a trajectory, computing missing segment distances when necessary.

Usage: Called by the Dubins solver and trajectory-analysis routines.

5) *traj_calctraj_angdist()*: **Purpose:** Computes each segment's heading change and horizontal distance for straight and turning motion.

Usage: Called by *Traj_HDist()*, *Traj_Calc3D()*, and Dubins-path evaluation.

6) *traj_calcangdist()*: **Purpose:** Computes the signed angular distance between two headings for clockwise or counterclockwise motion.

Usage: Called by *traj_semiplan()* and other turn-geometry routines.

7) *traj_semiplan()*: **Purpose:** Determines whether the clockwise angular separation between two headings lies within the opposite semicircle.

Usage: Auxiliary trajectory-geometry utility.

8) *Traj_Calc3D()*: **Purpose:** Propagates altitude through the trajectory using each segment's horizontal distance and flight-path angle.

Usage: Called after Dubins geometry is computed to obtain three-dimensional waypoint altitudes.

9) *Traj_ToPlot()*: **Purpose:** Converts a trajectory into a dense array of positions by sampling turns at small heading increments.

Usage: Used for visualization and high-resolution trajectory inspection.

10) *Traj_Print()*: **Purpose:** Writes trajectory waypoints and segment attributes to a formatted text or CSV stream.

Usage: General debugging and reporting utility.

11) *Traj_LayerTime()*: **Purpose:** Computes the time spent by a trajectory within a specified altitude layer.

Usage: Used for altitude-band exposure or trajectory-performance analysis.

12) *traj_file()*: **Purpose:** Samples the trajectory over time and writes position, velocity, heading, flight-path angle, lift coefficient, and bank-angle data to *traj.dat*.

Usage: Used to generate trajectory data for simulation or external post-processing.

IV. CONFLICT-AWARE TRAJECTORY PLANNER OUTPUT FILES

This section describes the files generated in the *out* directory and the meaning of each column. Files containing *sim* in their names are omitted because they are intended for simulation-specific interfaces. Unless stated otherwise, each row corresponds to one trajectory sample, search state, or planning case.

A. *ac1m.cfg*

This file is a copy of the runtime configuration associated with the current planning case. The initial, goal, and touchdown states are updated to reflect the landing site evaluated by the planner.

B. *path.csv*

This file contains the search-based solution path. The altitude values are recomputed using the corrected allocation of straight-flight and half-turn altitude losses.

Column 1: Latitude [deg].

Column 2: Longitude [deg].

Column 3: Corrected altitude [ft].

Column 4: Heading [deg].

Column 5: Time from the beginning of the search trajectory [s].

C. *actions.csv*

This file contains the discrete actions associated with the search-based solution path.

Column 1: Commanded course change, $\Delta\chi$ [deg].

Column 2: Action segment length [ft].

Column 3: Straight-flight glide path angle [deg].

Column 4: Turning-flight glide path angle [deg].

D. *finalturn.csv*

This file contains the final Dubins maneuver connecting the search-based path to the runway-aligned approach. The trajectory is sampled at one-second intervals.

Column 1: Latitude [deg].

Column 2: Longitude [deg].

Column 3: Altitude [ft].

Column 4: Heading [deg].

Column 5: Groundspeed [kts].

Column 6: Time from the beginning of the final turn [s].

E. *best_dubins.csv*

This file contains the time-sampled minimum-risk Dubins trajectory used as either a benchmark or fallback solution. When an S-turn is required, the samples from its two connected Dubins paths are combined into one continuous trajectory.

Column 1: Latitude [deg].

Column 2: Longitude [deg].

Column 3: Altitude [ft].

Column 4: Heading [deg].

Column 5: Time from the beginning of the Dubins trajectory [s].

F. *airspaceRisk0.csv*

This file contains the dynamic-airspace risk samples evaluated along the search-based trajectory, including its final Dubins turn. The suffix 0 denotes the search-based solution.

Column 1: Ownship latitude [deg].

Column 2: Ownship longitude [deg].

Column 3: Ownship altitude [ft].

Column 4: Ownship groundspeed [kts].

Column 5: Maximum instantaneous CPA risk, J , over all intruders.

Column 6: Cumulative integral of J .

G. *airspaceRisk1.csv*

This file contains the dynamic-airspace risk samples evaluated along the minimum-risk Dubins trajectory. The suffix 1 denotes the Dubins solution.

Column 1: Ownship latitude [deg].
Column 2: Ownship longitude [deg].
Column 3: Ownship altitude [ft].
Column 4: Ownship groundspeed [kts]. In the current implementation, this field may be reported as NaN for Dubins samples.
Column 5: Maximum instantaneous CPA risk, J , over all intruders.
Column 6: Cumulative integral of J .

H. conflicts0.csv

This file logs dynamic-traffic conflicts detected along the search-based trajectory. A row is written whenever the CPA risk for an intruder is greater than zero.

Column 1: Time index [s].
Column 2: Ownship latitude [deg].
Column 3: Ownship longitude [deg].
Column 4: Ownship altitude [ft].
Column 5: Ownship course [deg].
Column 6: Intruder latitude [deg].
Column 7: Intruder longitude [deg].
Column 8: Intruder altitude [ft].
Column 9: Intruder course [deg].
Column 10: Intruder identifier.
Column 11: Instantaneous CPA risk, J .

I. conflicts1.csv

This file has the same format as *conflicts0.csv*, but records conflicts along the minimum-risk Dubins trajectory.

Column 1: Time index [s].
Column 2: Ownship latitude [deg].
Column 3: Ownship longitude [deg].
Column 4: Ownship altitude [ft].
Column 5: Ownship course [deg].
Column 6: Intruder latitude [deg].
Column 7: Intruder longitude [deg].
Column 8: Intruder altitude [ft].
Column 9: Intruder course [deg].
Column 10: Intruder identifier.
Column 11: Instantaneous CPA risk, J .

J. census_data_0.csv

This file contains the population-density samples used to evaluate ground risk along the search-based trajectory. The suffix 0 denotes the search-based path.

Column 1: Latitude [deg].
Column 2: Longitude [deg].
Column 3: Altitude [ft].
Column 4: Normalized population density.
Column 5: Cumulative population-risk integral.

K. census_data_1.csv

When generated, this file contains the population-density samples used to evaluate the Dubins trajectory. Its columns are identical to those of *census_data_0.csv*.

Column 1: Latitude [deg].
Column 2: Longitude [deg].
Column 3: Altitude [ft].
Column 4: Normalized population density.
Column 5: Cumulative population-risk integral.

L. explored.csv

This file contains the search states transferred to the explored, or closed, set.

Column 1: State latitude [deg].
Column 2: State longitude [deg].
Column 3: State altitude [ft].
Column 4: State heading [deg].
Column 5: Cumulative path cost, g .
Column 6: Heuristic cost, h .
Column 7: Total search priority, f .

M. explored_actions.csv

This file contains the action associated with each state in *explored.csv*. Rows in the two files correspond by index.

Column 1: Commanded course change [deg].
Column 2: Action segment length [ft].
Column 3: Straight-flight glide path angle [deg].

N. runtime.csv

This file records the search outcome and runtime for each ranked landing site attempted during a planning case.

Column 1: Landing-site rank index.
Column 2: Landing-site identifier.
Column 3: Search exit flag.
Column 4: Search runtime [ms].

O. log.csv

This file is the cumulative case-level summary log. One row is appended for each planning case.

Column 1: Planner exit flag.
Column 2: Initial latitude [deg].
Column 3: Initial longitude [deg].
Column 4: Initial altitude [ft].
Column 5: Initial heading [deg].
Column 6: Search-path ground risk.
Column 7: Search-path airspace risk.
Column 8: Search-path glide-angle modification [deg].
Column 9: Total search runtime [ms].
Column 10: Search-path risk-evaluation runtime [ms].
Column 11: Dubins-path ground risk.
Column 12: Dubins-path airspace risk.
Column 13: Dubins-path glide-angle modification [deg].
Column 14: Total Dubins-planning runtime [ms].
Column 15: Dubins risk-evaluation runtime [ms].
Column 16: Identifier of the landing site selected by the planner.
Column 17: Identifier of the highest-ranked landing site.

V. CONFLICT RESOLUTION SOFTWARE

This section documents the Conflict Resolution software, which implements the *Avoid* stage of the Plan-and-Avoid (PAA) framework. Whereas the *Plan* stage generates a priority trajectory that minimizes avoidable conflicts, the *Avoid* stage resolves the remaining conflicts by generating cooperative resolution advisories for surrounding aircraft while preserving the selected priority trajectory. The software operates on the trajectory and conflict information produced by the planning stage and outputs conflict-resolved advisory solutions together with supporting logs for analysis and visualization.

A. Directory Structure

The overall directory organization follows the same modular design as the planning software. Reusable functionality is encapsulated in independent library modules, while the application entry point and generated advisories are organized into separate directories.

1) *advisory*: The *advisory* directory stores the generated resolution-advisory solutions for both the search-based and Dubins priority trajectories. These files contain the computed advisory trajectories together with log files that record advisory details, conflict information, and optimization results for post-processing and analysis.

2) *lib*: The *lib* directory contains the reusable software modules used by the conflict resolution application. As in the planning software, each module follows a common directory organization consisting of *src*, *include*, *obj*, and, where required, *data* subdirectories.

3) *src*: The *src* directory contains the application entry point. The main source file loads the planning results generated by the *plan* module, invokes the conflict-resolution algorithms, and writes the resulting advisories and diagnostic information to the *advisory* directory.

B. Build Configuration

The *avoid* application is compiled using the top-level Makefile. C++ source files are compiled with `clang++` under the C++17 standard, while C source files are compiled with `clang` under the C11 standard. The resulting object files are linked into the *avoid* executable.

The project is built using

```
make
```

The generated object files and executable are removed using

```
make clean
```

1) *Compiler and Language Settings*: **CXX**: Path to the C++ compiler.

CC: Path to the C compiler.

CXXFLAGS: Enables C++17, compiler warnings, and optional sanitizer flags.

CFLAGS: Enables C11, compiler warnings, and optional sanitizer flags.

SANFLAGS: Optional sanitizer options. This variable may be left empty for normal builds or defined for debugging, for example:

```
SANFLAGS = -fsanitize=address,undefined -g
```

2) *Header Search Paths*: The **CPPFLAGS** variable defines the public-header directories used during compilation. These include the top-level *include* directory and the library headers under *lib/*/include*. It also specifies the external include directories for **NLOpt** and **libspatialindex**.

The internal project paths require no modification if the supplied directory structure is preserved. Users must update the following external paths when the dependencies are installed elsewhere:

- /opt/homebrew/Cellar/nlopt/2.10.0/
include
- /opt/homebrew/Cellar/spatialindex/2.1.
0/include

The *lib/resolution/include* path appears twice and one duplicate entry may be removed without changing the build behavior.

3) *External Libraries*: The **LDFLAGS** variable links the following external libraries:

NLOpt: Nonlinear optimization library used by the advisory optimizer.

libspatialindex: Spatial-indexing library used by the R*-tree wrapper.

Shapelib: Provides shapefile and DBF support through *libshp*.

The current paths correspond to a macOS Homebrew installation. Users must update

- /opt/homebrew/Cellar/nlopt/2.10.0/lib
- /opt/homebrew/Cellar/spatialindex/2.1.
0/lib
- /usr/local/lib

to the matching library directories on their system. The runtime search path

```
-Wl,-rpath,/usr/local/lib
```

must also be changed when *libshp* is installed in another location.

4) *Source Files*: The C++ sources include the application entry point and the conflict-resolution modules:

- *src/avoid.cpp*
- *lib/resolution/src/advisory.cpp*
- *lib/resolution/src/conflict.cpp*
- *lib/resolution/src/traffic.cpp*

The C sources provide shared geodesic, Dubins, trajectory, mathematical, and air-traffic functionality:

- *lib/geo/src/geo.c*
- *lib/dubins/src/dubins.c*
- *lib/traj/src/traj.c*
- *lib/math_utils/src/math_utils.c*
- *lib/airtraffic/src/airtraffic.c*

Object files are generated automatically under *obj* while preserving the source-directory hierarchy. The `mkdir -p` command creates the required object subdirectories during compilation.

When a source file is added, removed, or relocated, the corresponding entry in **CPP_SRC** or **C_SRC** must be updated. No separate compilation rule is required because the pattern rules automatically generate the associated object files.

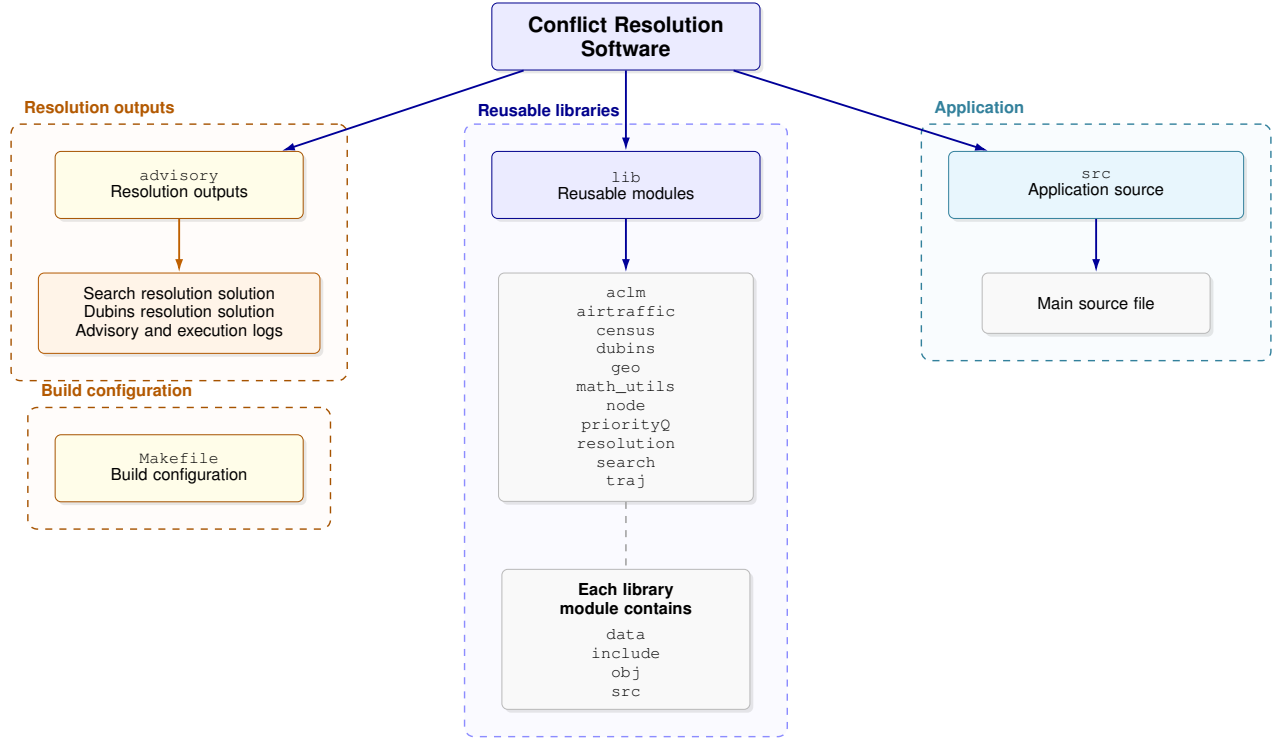


Fig. 3. Conflict-resolution software directory structure. The advisory directory stores the generated resolution solutions and corresponding logs, while reusable functionality is organized under lib and the application entry point is contained in src.

5) *Build Targets*: **all**: Builds the avoid executable.
clean: Deletes the obj directory and the executable.
run: Builds the executable when necessary and then runs ./avoid.

VI. RESOLUTION ADVISORY SOURCE CODE REFERENCE

This section documents the source files that implement the Conflict Resolution Advisory System. Figure 4 summarizes the high-level execution flow and interactions among the principal modules, from conflict log initialization and traffic data parsing to the sequential evaluation of speed, altitude, and lateral tactical advisories, ending with separation risk verification and output generation. The remainder of the section describes each source file, summarizing its principal data structures and functions, its role in the overall advisory workflow, and its primary points of use.

This section documents the source files that implement the Conflict Resolution Advisory System. Figure 4 summarizes the high-level execution flow and interactions among the principal modules, from conflict log initialization and traffic data parsing to the sequential evaluation of speed, altitude, and lateral tactical advisories, ending with separation risk verification and output generation. The remainder of the section describes each source file, summarizing its principal data structures and functions, its role in the overall advisory workflow, and its primary points of use.

A. avoid.cpp

The file avoid.cpp implements the top-level application entry point and driver for the conflict-resolution subsystem.

It verifies and pairs emergency trajectory files with their corresponding conflict logs for both Search and Dubins planner solutions, configures the optimization engine, runs the iterative resolution loops, updates aggregate statistical tracking counters, and writes formatted logs and spatial resolution advisories.

1) *configureConflictSolver()*: **Purpose**: Initializes optimization options and workspace parameters for a conflict resolution instance, including populating the prohibited-airspace geographic data arrays, setting the population size, imposing evaluation time limits, defining the planetary reference ellipsoid model, and turning off internal layout diagnostic printouts. **Usage**: Called by processSolution() during initialization of a localized Conflict object instance.

2) *emergencyPath()*: **Purpose**: Constructs the absolute filesystem path pointing to an emergency trajectory risk output file by appending a hardcoded filename suffix to the active planner results directory. **Usage**: Called by solutionExists(), reportIncompleteSolution(), and processSolution() to locate trajectory data.

3) *conflictLogPath()*: **Purpose**: Constructs the absolute filesystem path pointing to a binary or character-separated conflict history log file by appending a hardcoded filename suffix to the active planner results directory. **Usage**: Called by solutionExists(), reportIncompleteSolution(), and processSolution() to locate conflict log files.

4) *solutionExists()*: **Purpose**: Evaluates whether an execution pipeline context contains a complete dataset by checking for the simultaneous structural presence of both

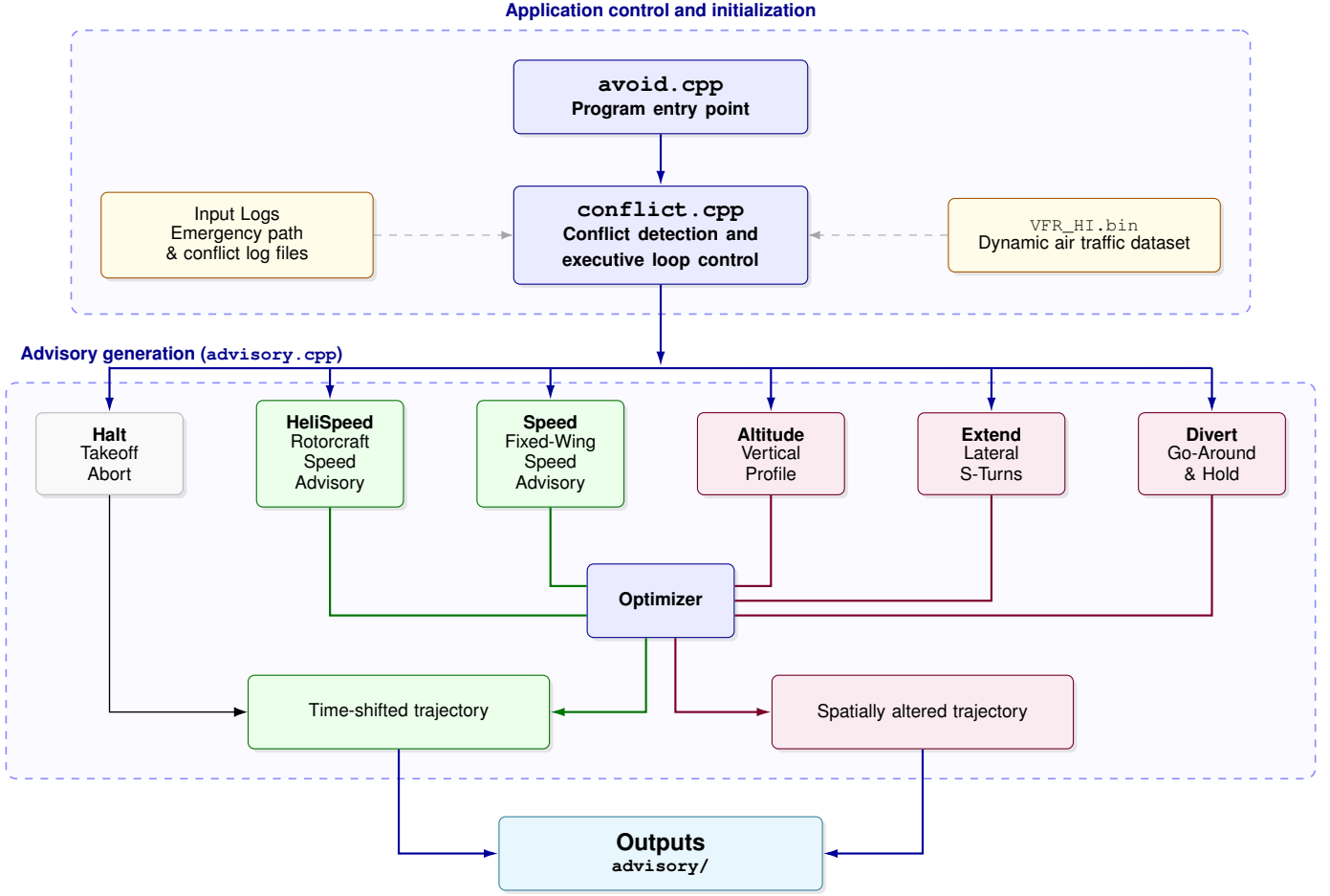


Fig. 4. High-level real-time tactical conflict resolution advisory execution flow. Individual profile optimization loops and lateral geometric detour subclasses are executed out of `advisory.cpp` and verified using common spatial separation and background risk algorithms.

a solution's path geometry and its recorded airspace traffic violations. **Usage:** Called within the core execution loop inside `main()` to determine whether to process or skip a specific planning solution category.

5) `reportIncompleteSolution()`: **Purpose:** Scans a targeted planner output folder and issues standard error console warnings if it identifies an asynchronous state where exactly one of the two mandatory trajectory files is missing. **Usage:** Called dynamically at the beginning of the evaluation loop within `main()` prior to executing any solver procedures.

6) `updateFlagCounts()`: **Purpose:** Iterates through a collection of returned exit state codes from a solver run, validates that the indices do not exceed bounds, and bumps corresponding accumulator tallies used for final summary metrics. **Usage:** Called by `processSolution()` immediately following an active trajectory optimization run.

7) `processSolution()`: **Purpose:** Orchestrates conflict assessment for an isolated path workflow by creating a local, stateless context object, verifying conflict counts, invoking the core iterative solver, and routing output data into localized case and resolution output tables. **Usage:** Called within the loop container in `main()` for every valid, fully formed candidate plan detected.

8) `printResolutionSummary()`: **Purpose:** Formats and prints a structured summary grid tracking the aggregate distribution of final resolution action strategies, such as extensions, speed holds, or initial collision states. **Usage:** Called by `main()` as a terminal reporting utility before exiting with a success token.

9) `main()`: **Purpose:** Serves as the primary operational entry point for the avoidance module. It maps relative deployment working directories, verifies file access handles for tracking logs, initializes structure tables, loops over existing planner profiles, handles exceptional failures, and prints output summaries. **Usage:** System execution entry point.

B. `advisory.cpp`

The file `advisory.cpp` implements various multi-aircraft conflict resolution strategies (advisories) designed to dynamically modify intruder trajectories and guarantee separation. It includes logic for halting takeoffs, optimizing helicopter and fixed-wing ground speeds, altering vertical flight profiles, generating lateral S-turn paths via Dubins tracking, and rerouting conflicting traffic to designated holding fixes. Each resolution subsystem balances ownship conflict constraints with domino-effect traffic risks using the `nlopt` optimization engine.

1) *Halt::isTakeoff()*: **Purpose:** Determines if a conflicting aircraft is currently in its takeoff phase by scanning historical flight telemetry for a sustained vertical climb rate exceeding a specific threshold over a given time window. **Usage:** Called internally by *Halt::run()* to verify if the aircraft is a valid candidate for a takeoff halt advisory.

2) *Halt::run()*: **Purpose:** Executes the takeoff halt strategy by intercepting an aircraft prior to departure, generating an unachievable trajectory filled with NaN coordinates to signal an immediate abort command, and writing the transaction logs to disk. **Usage:** Called by the top-level subsystem loop when responding to departure-phase traffic conflicts.

3) *Helispeed::speedCommand()*: **Purpose:** Modifies a helicopter's ground speed profile over an advisory window by applying an acceleration or deceleration ramp to a target offset, holding that speed, and recovering back to the planned schedule while enforcing physical operational limits. **Usage:** Called by *Helispeed::timeShiftTrajectory()* to calculate the target helicopter ground speed sequence.

4) *Helispeed::timeShiftTrajectory()*: **Purpose:** Re-interpolates an intruder helicopter's spatial coordinates over time by matching its cumulative traveled distance profile against the newly computed ground speed profile. **Usage:** Called by optimization constraints and tracking routines to project the physical path under speed regulation.

5) *Helispeed::nonlcon()* / *Helispeed::nonlconDomino()*: **Purpose:** Defines the nonlinear inequality constraints for the optimization loop, ensuring that the modified helicopter trajectory maintains a safe ownship separation window and does not trigger secondary ("domino") traffic conflicts. **Usage:** Assigned as callback functions to the *nlopt* optimization structure during parameter tuning.

6) *Helispeed::cost()*: **Purpose:** Computes a normalized quadratic objective cost that penalizes large ground speed deviations, late advisory onset times, and long holding durations to find the least disruptive resolution strategy. **Usage:** Passed directly to the *nlopt* engine as the core minimization objective.

7) *Helispeed::optimize()*: **Purpose:** Configures and runs local or global optimization searches (e.g., *GN_ISRES* or *GN_MLSL*) using *nlopt* to discover the optimal speed adjustment magnitude, start timestamp, and hold duration. **Usage:** Called by *Helispeed::run()* to solve for the best helicopter parameters.

8) *Helispeed::run()*: **Purpose:** Acts as the primary entry point for the helicopter speed adjustment strategy, handling configuration setup, calling the optimization sequence, assessing feasibility, and logging performance metrics. **Usage:** Called by the main conflict resolution driver to evaluate helicopter speed control.

9) *Speed::groundToAirspeed()* / *Speed::airToGroundspeed()*: **Purpose:** Performs vector transformations between ground tracks and aerodynamic airspeeds to account for ambient wind velocity and direction. **Usage:** Used inside *Speed::speedCommand()* to ensure that proposed

ground speed changes do not violate structural or aerodynamic airspeed boundaries.

10) *Speed::speedCommand()*: **Purpose:** Generates a customized fixed-wing ground speed modification profile utilizing acceleration and deceleration transitions, a steady-state hold window, and an airspeed envelope guardrail. **Usage:** Called by *Speed::timeShiftTrajectory()* during fixed-wing resolution steps.

11) *Speed::cumulativeDistance()*: **Purpose:** Computes the cumulative distance along a trajectory profile at each discrete step using numerical trapezoidal integration. **Usage:** Called by *Speed::timeShiftTrajectory()* to generate distance references for path re-interpolation.

12) *Speed::timeShiftTrajectory()*: **Purpose:** Adjusts fixed-wing trajectory timing by mapping current positions along a path using the integrated distance profile generated under modified speed commands. **Usage:** Used within objective functions, constraints, and logging loops to output the complete adjusted trajectory.

13) *Speed::nonlcon()* / *Speed::nonlconDomino()* / *Speed::cost()* / *Speed::optimize()* / *Speed::run()*: **Purpose:** Forms the optimization, evaluation, and execution pipeline for fixed-wing aircraft speed control, matching the functional behavior of the helicopter variants while incorporating fixed-wing specific performance envelopes. **Usage:** Executed by the top-level driver when managing fixed-wing separation conflicts.

14) *Altitude::verticalRate()* / *Altitude::isClimb()*: **Purpose:** Calculates an aircraft's discrete vertical rate of climb or descent over a specified time window and classifies the phase as a climb, level flight, or descent. **Usage:** Called inside *Altitude::shiftVerticalProfile()* to establish the base vertical trends of the aircraft.

15) *Altitude::shiftVerticalProfile()*: **Purpose:** Creates a modified vertical flight track by injecting a temporary step-change in altitude using defined climb/descent angles, a hold phase, and a recovery segment while enforcing minimum safe altitudes. **Usage:** Called by vertical optimization constraints and trajectory generation tools to construct altered profiles.

16) *Altitude::nonlcon()* / *Altitude::nonlconDomino()* / *Altitude::cost()* / *Altitude::optimize()* / *Altitude::run()*: **Purpose:** Implements the standard optimization framework for vertical separation adjustments, tuning parameters such as altitude deviation magnitude (δH), hold time, departure angle (γ_0), recovery angle (γ_1), and implementation timestamp (t_0). **Usage:** Run by the main loop to handle vertical separation requests.

17) *Extend::interpPos()*: **Purpose:** Performs smooth linear interpolation between two spatial points while applying a wrap-safe calculation to manage angular heading crossings through 360°. **Usage:** Used when resampling waypoints during lateral trajectory updates.

18) *Extend::buildSTurnPath()* / *Extend::generateSTurnPath()*: **Purpose:** Calculates a parameterized, lateral S-turn detour using multi-segment Dubins path structures based on localized turn radii and custom heading changes (θ). **Usage:** Called by *Extend::divert()* and optimization handlers to map alternative lateral tracks.

19) *Extend::DubinsCoordinates()*: **Purpose:** Generates discrete coordinate points at a constant time step along the geodetic segments of a computed Dubins path. **Usage:** Called inside *Extend::divert()* to translate geometric curves into actionable trajectory tables.

20) *Extend::divert()*: **Purpose:** Creates a full composite track by substituting a segment of an aircraft's original flight plan with an S-turn detour path while validating that the track clears prohibited airspaces. **Usage:** Called during constraint evaluations and successful path exports.

21) *Extend::nonlcon()* / *Extend::nonlconDomino()* / *Extend::cost()* / *Extend::optimize()* / *Extend::run()*: **Purpose:** Orchestrates the search loop for lateral S-turn diversions, optimizing the turn geometry (θ) alongside execution start and finish timestamps (t_0, t_f). **Usage:** Invoked by the primary controller to resolve lateral spatial conflicts.

22) *Divert::computePath()*: **Purpose:** Determines a valid Left-Straight-Left (LSL) or Right-Straight-Right (RSR) Dubins path to route an aircraft from a conflict area directly to a designated holding point. **Usage:** Called inside *Divert::pathToHold()* to lay out the recovery route geometry.

23) *Divert::pathToHold()*: **Purpose:** Builds a complete trajectory sequence routing an intruder safely to a holding fix, monitoring boundary violations and flatlining the vertical profile upon reaching the holding altitude. **Usage:** Called by optimization functions to project path variations when diverting to a holding pattern.

24) *Divert::nonlcon()* / *Divert::nonlconDomino()* / *Divert::cost()* / *Divert::optimize()*: **Purpose:** Configures the optimization variables (flight path angle γ and onset time t_0) for aircraft rerouting while validating that the result satisfies separation constraints. **Usage:** Executes internally within the global evaluation loop of the holding subsystem.

25) *Divert::run()*: **Purpose:** Manages the holding pattern diversion strategy by evaluating proximity to available holding fixes, optimizing flight angles for the closest option, and exporting the final trajectory. **Usage:** Run as a fallback or tactical mitigation choice during severe spatial conflicts.

C. *conflict.cpp*

The file *conflict.cpp* implements the conflict detection, risk assessment, and resolution execution pipeline for aircraft facing spatial conflicts with dynamic traffic. It manages traffic logs, calculates horizontal and vertical separation distances, estimates collision risks using Closest Point of Approach (CPA) models, and sequences through the available operational advisories to resolve traffic threats.

1) *Conflict::resolve()*: **Purpose:** Coordinates the core resolution workflow by loading dynamic air traffic, filtering unique intruder threats, checking if the scenario initialized in an unavoidable conflict state, and sequentially evaluating advisory strategies (takeoff halt, speed modifications, altitude changes, extends, or holding loops) until it secures a safe trajectory. **Usage:** Called by the primary simulation driver to find and log resolution paths for a specific flight scenario.

2) *Conflict::getPathRuntime()*: **Purpose:** Opens the emergency landing path generation log file to parse, extract, and convert the core execution runtime recorded in the final entry. **Usage:** Used to log initial path planning performance characteristics prior to handling traffic conflicts.

3) *Conflict::loadEmergencyPath()*: **Purpose:** Parses a geometric CSV flight profile and extracts coordinates, headings, and sequential timestamps to populate the internal emergency tracking path vector. **Usage:** Called to load the baseline flight path of the ownship during an active emergency event.

4) *Conflict::loadConflictLog()*: **Purpose:** Reads a logged conflict file to populate the internal database with historical or predicted separation violations between the ownship and dynamic traffic. **Usage:** Invoked at the start of the conflict processing pipeline to determine which spatial conflicts require mitigation.

5) *Conflict::conflictInstance()*: **Purpose:** Retrieves a single structural conflict log row from the internal tracking database using its direct index. **Usage:** Used inside evaluation routines to inspect specific coordinates and tracking variables of a conflict.

6) *Conflict::unique()*: **Purpose:** Sweeps the complete conflict log to filter and compile the very first instance of a conflict for each unique intruder aircraft ID. **Usage:** Called during resolution setup to isolate individual target aircraft and avoid processing redundant conflict rows.

7) *Conflict::retimeEmergencyPathBySpeed()*: **Purpose:** Resamples the geometric coordinates of an emergency path based on a target ground speed and discrete time step, interpolating spatial locations to build a newly timed tracking timeline. **Usage:** Used to dynamically alter and evaluate ownship tracking schedules when testing different speed profiles.

8) *Conflict::computeSeparation()*: **Purpose:** Computes the horizontal and vertical distances between ownship and intruder coordinates while subtracting safety margins assigned to sensor errors and operational airspace boundaries. **Usage:** Called throughout risk, margin, and optimization calculations to check spatial clearance.

9) *Conflict::CPACost()*: **Purpose:** Translates raw horizontal and vertical clearance margins into a normalized mathematical penalty score using smooth sigmoid functions when aircraft violate minimum separation thresholds. **Usage:** Used as the mathematical basis for risk estimation and trajectory optimization.

10) *Conflict::trafficRisk()*: **Purpose:** Quantifies the domino-effect risk of an aircraft's altered trajectory by evaluating and integrating the worst-case CPA costs against all

other active dynamic traffic flights. **Usage:** Called by optimization constraints to ensure an advisory does not inadvertently cause a secondary conflict with ambient traffic.

11) *Conflict::ownshipMarginSingle()*:

Purpose: Identifies the smallest physical clearance margin across the overlapping tracking profiles of the ownship and a specific intruder aircraft. **Usage:** Used for basic pairwise trajectory evaluation.

12) *Conflict::ownshipMarginRobustTimeWindow()*:

Purpose: Estimates safe clearance margins across a flexible time window to protect the ownship against velocity uncertainties along its flight path. **Usage:** Used for robust safety checks when handling velocity drift.

13) *Conflict::ownshipMargin()*: **Purpose:** Traverses the entire active traffic database to find the absolute minimum physical clearance margin encountered by the ownship across all points in its flight profile. **Usage:** Used to evaluate total situational clearance for the ownship.

14) *Conflict::ownshipRisk()*: **Purpose:** Calculates both the cumulative integrated risk and worst-case risk score for the ownship by computing CPA costs against all surrounding dynamic traffic flights. **Usage:** Called to assess and log final post-resolution safety performance metrics.

15) *printCasesHeader()* / *printCaseRow()*:

Purpose: Formats and writes the scenario summary structure (including success rates, total runtimes, and residual risks) out to a case log file stream. **Usage:** Executed at the end of a conflict resolution batch to output scenario performance metrics.

16) *printResolutionsHeader()* /

printResolutionRow(): **Purpose:** Formats and saves detailed advisory parameters, including optimization flags, onset times, specific speed offsets, altitude steps, turn angles, and optimization variables. **Usage:** Called following an advisory calculation to record granular resolution steps to disk.

17) *printOptional()* /

printOptionalBool(): **Purpose:** Extracts and writes values from optional variable containers into an active file log stream if valid data exists. **Usage:** Helper layout functions used inside log generation routines.

18) *loadLogCSV()*: **Purpose:** Opens a target planner log file and extracts the final exit status flags across all listed scenario rows. **Usage:** Used to reload or verify operational solution flags from historical log files.

D. traffic.cpp

The file *traffic.cpp* handles the parsing, management, and physical property estimation of dynamic air traffic. It provides utilities to stream binary flight records from disk, classify conflicting aircraft types, and approximate operational turn limits based on current telemetry and vehicle categories.

1) *read_exact()*: **Purpose:** Reads a specific byte length matching a target data type directly from an input file stream and throws a runtime exception with a custom message if the file operation fails. **Usage:** Used internally as a helper routine inside binary file parsers to safely extract metadata and flight parameters.

2) *Traffic::loadDynamicTraffic()*: **Purpose:** Opens a binary air traffic file, validates its custom format header ("ATDIBIN"), reads file statistics, and resamples and stores sequential trajectory data tables across all available flights into an internal data array. **Usage:** Called during conflict manager initialization to populate the active simulator environment with background traffic trajectories.

3) *Traffic::type()*: **Purpose:** Looks up and returns the active aircraft type enum for a specific vehicle given its unique tracking identification index. **Usage:** Used by conflict resolution algorithms to determine which aircraft-specific performance models and limits to apply.

4) *Traffic::typeString()*: **Purpose:** Converts an aircraft's internal numeric classification type into a concise, human-readable string shortcode, mapping vehicles to "GA" (General Aviation), "TJ" (Turbojet), "H" (Helicopter), or "XX" (Unknown). **Usage:** Called within formatting routines to write readable category tags into the final resolution logs.

5) *Traffic::turnRadius()*: **Purpose:** Approximates a target aircraft's operational turn radius at a specific timestamp by checking its instantaneous ground speed and computing a physics-based radius using a 30-degree bank angle for helicopters or a standard 3-degree-per-second turn rate for fixed-wing aircraft. **Usage:** Invoked by lateral tracking modules to bound alternative trajectory designs within real-world aerodynamic performance constraints.

VII. RESOLUTION ADVISORY OUTPUT FILES

A. 00_cases.csv

This file provides a high-level summary of each simulated conflict scenario, tracking overall processing success rates, computational execution runtimes, and residual safety risks across the evaluation batch.

Column 1: Case identification number (*case_id*).

Column 2: Operational scenario processing status flag (*case_status*).

Column 3: Total count of unique intruder aircraft conflicts detected within the scenario (*n_conflicts*).

Column 4: Count of traffic conflicts successfully cleared by an advisory (*n_resolved*).

Column 5: Count of traffic conflicts where no mathematically valid resolution could be found (*n_failed*).

Column 6: Total wall-clock runtime spent solving all conflicts within the case [ms] (*total_runtime_ms*).

Column 7: Integrated cumulative CPA risk profile experienced by the ownship over the advisory window (*adv_ownship_int_cpa*).

Column 8: Absolute worst-case peak risk index experienced by the ownship post-resolution (*adv_ownship_worst_cpa*).

B. 00_resolutions.csv

This detailed log captures granular optimization control parameters, specific maneuver profiles, and localized traffic risk metrics for every individual intruder resolution attempt.

Column 1: Parent case identification number (*case_id*).

Column 2: Unique tracking identification index of the conflicting intruder aircraft (`intruder_id`).

Column 3: Performance category classification type of the intruder (`intruder_type`).

Column 4: Core resolution strategy subsystem invoked to handle the threat (`advisory_type`).

Column 5: Operational mitigation category exit flag code (`advisory_exit_flag`).

Column 6: Numerical status code returned directly by the core NLOpt optimization engine (`optimization_exit_flag`).

Column 7: Scheduled start timestamp where the advisory command begins execution [s] (`onsetTime_s`).

Column 8: Originally predicted timestamp of closest approach for the traffic conflict [s] (`conflictTime_s`).

Column 9: Integrated background traffic risk along the original unmitigated flight path (`plan_traffic_int_cpa`).

Column 10: Integrated background traffic risk along the newly generated advisory flight path (`adv_traffic_int_cpa`).

Column 11: Peak worst-case collision risk to background traffic along the original unmitigated flight path (`plan_traffic_worst_cpa`).

Column 12: Peak worst-case collision risk to background traffic along the newly generated advisory flight path (`adv_traffic_worst_cpa`).

Column 13: Optimized target airspeed or ground speed profile magnitude [kts] (`V_kts`).

Column 14: Solved start time offset for executing the ground speed modification ramp [s] (`t0_s`).

Column 15: Solved steady-state time duration window for maintaining the adjusted speed profile [s] (`T_s`).

Column 16: Solved step-change vertical altitude deviation offset [ft] (`H_ft`).

Column 17: Solved execution start timestamp for the vertical maneuvering profile [s] (`t0_s`).

Column 18: Solved duration window spent at the temporary separation altitude hold phase [s] (`T_s`).

Column 19: Optimized entry flight path angle for the initial climb or descent maneuver [deg] (`gamma0_deg`).

Column 20: Optimized recovery flight path angle used to return to the nominal route profile [deg] (`gamma1_deg`).

Column 21: Optimized lateral heading deviation turn adjustment angle [deg] (`theta_deg`).

Column 22: Solved execution start timestamp for the extension maneuver [s] (`t0_s`).

Column 23: Solved recovery finish timestamp for the extension maneuver [s] (`tf_s`).

Column 24: Calculated spatial track distance of the lateral detour path [NM] (`length_nm`).

Column 25: Optimized entry flight path angle chosen during a holding pattern diversion route [deg] (`gamma_deg`).

Column 26: Total route length tracking from the conflict zone out to the designated holding fix [NM] (`length_nm_hold`).

Column 27: Solved execution start timestamp for navigating to the holding fix [s] (`t0_s`).

Column 28: Database lookup index matching the targeted geographic recovery holding fix (`hold_idx`).

Column 29: Mathematical value of the first nonlinear inequality separation constraint (g_1^*) at solver termination

(`g1_star`).

Column 30: Mathematical value of the second nonlinear inequality domino-conflict constraint (g_2^*) at solver termination (`g2_star`).

Column 31: Final objective function cost value (f^*) achieved by the optimization loop (`f_star`).

Column 32: Binary safety feasibility flag where 1 denotes all separation criteria were strictly satisfied (`feasible`).

Column 33: Total computational execution runtime of the isolated advisory subsystem loop [ms] (`runtime_ms`).

C. `<intruderID>_trajectory.csv`

Located within the `Dubins` and `Search` directories, these files represent conflict resolution advisories generated for each corresponding contingency landing solution. This specific file contains the second-by-second time-series trajectory comparison for an individual intruder aircraft, contrasting its original nominal flight path with the modified trajectory generated by the lateral or speed advisory system at a constant sampling frequency of 1 Hz.

Column 1: Nominal aircraft latitude [deg].

Column 2: Nominal aircraft longitude [deg].

Column 3: Nominal aircraft altitude [ft].

Column 4: Nominal aircraft heading [deg].

Column 5: Nominal aircraft groundspeed [kts].

Column 6: Advised aircraft latitude [deg].

Column 7: Advised aircraft longitude [deg].

Column 8: Advised aircraft altitude [ft].

Column 9: Advised aircraft heading [deg].

Column 10: Advised aircraft groundspeed [kts].